# Onyx — NetKit reference

**Onyx · multi-process OS on Raspberry Pi 4**

*The reference of **NetKit** (`SD:/lib/netkit.so`, `user/Kits/netkit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: The Kits.*

## Contents


1. What it is
2. Using it
3. Index
4. `netkit/httpc.h`
5. `netkit/ftpfs.h`
6. `netkit/http.hpp`


---

## What it is

NetKit is the network for the programs: a small HTTP client, the FTP volumes, and the HTTP/1.1 class with its TLS transport (`netkit/http.hpp`, included apart: still a header with its code). The sockets themselves are AppKit's (`kapi_tcp_*`).

| | |
|---|---|
| Include | `#include "netkit/netkit.h"` |
| Link | `lib/netkit.imp.a` (C++) or `lib/netkit.imp_c.a` (C) |
| Library | `SD:/lib/netkit.so` — 28 entries in its table (`user/Kits/netkit/netkit.abi`, append-only) |
| Sources | `user/Kits/netkit/` |

## Using it

**A page fetched** (HTTP/1.0, no allocation: the caller gives the buffer):

```
#include "appkit/appkit.h"
#include "netkit/netkit.h"

static char buf[32 * 1024];

int main (void)
{
    http_response r;
    if (http_get ("http://example.com/", buf, sizeof buf, &r) < 0 || !r.ok)
    {
        ax_putln ("request failed");
        return 1;
```

```
    }
    kapi_stdout_write (r.body, (unsigned) r.body_len);
    return 0;
}
```

**A web API** (HTTP/1.1, headers, HTTPS with the TLS transport) — the `HttpClient` class:

```
#include "netkit/http.hpp"

static char buf[64 * 1024];
HttpClient http;
http.user_agent ("MyApp/1.0").accept ("application/json");
HttpResponse r = http.get ("http://api.example.com/status", buf, sizeof buf);
if (!r.is_error ()) { /* r.status, the body in buf */ }
```

**An FTP server as a volume**: once logged in, `FTP:host/dir/file` is a path like any other, for every program.

```
#include "netkit/netkit.h"

if (ftpfs_login ("ftp.example.com", "me", "secret", 1))      // 1: remembered
{
    void *f = kapi_open ("FTP:ftp.example.com/pub/readme.txt");
    /* kapi_read ... kapi_close */
}
```

**The known Wi-Fi networks** (`wifi.h`, 2026-10-09): `SD:/etc/wpa_supplicant.conf` read and written with every network kept (the one joined last first), joining one at once (the scan itself is AppKit's `kapi_wlan_scan`). The menu bar's Wi-Fi menu, the Wi-Fi applet and the console's Wi-Fi page share it:

```
struct wifi_known k[WIFI_KNOWN_MAX]; char cc[8];
int n = wifi_known_load (k, WIFI_KNOWN_MAX, cc, sizeof cc);   // the networks, the country
int r = wifi_join ("Maison", WLAN_SEC_WPA2, "the password");  // WIFI_OK (joining),
WIFI_SAVED (a reboot joins it), WIFI_E*
wifi_forget ("Voisin");
wifi_set_country ("BE");
```

The passwords are in clear text on the card (the radio needs them).

The raw sockets (`kapi_tcp_connect`, `kapi_tcp_send`…) are AppKit's; NetKit is where protocols built on them belong.

## Index

Everything the headers declare, in their order — the details are in each header's part below.

| Name | What it does | Header |
|---|---|---|
| `http_request` | Perform an HTTP request. | `httpc.h` |
| `http_get` | Perform an HTTP request. | `httpc.h` |
| `http_post` | Perform an HTTP request. | `httpc.h` |
| `ftpfs_site` | (a type) | `ftpfs.h` |
| `ftpfs_obf_hex` | a password -> its hexpass (XOR a fixed key, in hex: obfuscated, not | `ftpfs.h` |

| Name | What it does | Header |
|---|---|---|
| | encrypted) in out (cap bytes, cut to fit) | |
| `ftpfs_unobf_hex` | a hexpass (ftpfs_obf_hex) -> the password in out (cap bytes, cut to fit) | `ftpfs.h` |
| `ftpfs_parse_site` | Parse one file line | `ftpfs.h` |
| `ftpfs_format_site` | Format one file line (with '\n') | `ftpfs.h` |
| `ftpfs_text_fix` | A text file saved by a Windows editor | `ftpfs.h` |
| `ftpfs_load_sites` | The remembered servers, in file order. | `ftpfs.h` |
| `ftpfs_login` | The remembered servers, in file order. | `ftpfs.h` |
| `ftpfs_login_site` | The remembered servers, in file order. | `ftpfs.h` |
| `ftpfs_forget` | The remembered servers, in file order. | `ftpfs.h` |
| `HttpError` | Transport / protocol errors are reported as a NEGATIVE HttpResponse::status. | `http.hpp` |
| `slen` | The length of the string s (0 for a null pointer). | `http.hpp` |
| `lc` | c in lower case ('A'..'Z' only). | `http.hpp` |
| `cat` | Append s (or n bytes of s) to d[*o], never overflowing cap | `http.hpp` |
| `catn` | Append s (or n bytes of s) to d[*o], never overflowing cap | `http.hpp` |
| `cati` | Append s (or n bytes of s) to d[*o], never overflowing cap | `http.hpp` |
| `find` | Case-insensitive lookup of a header in [hdr, hdr+hlen). | `http.hpp` |
| `parse_int` | The decimal number at the start of [s, s+n) (its leading digits, no sign) -> its value, 0 if there is no | `http.hpp` |

| Name | What it does | Header |
|---|---|---|
| | digit. | |
| `contains_ci` | Case-insensitive | `http.hpp` |
| `dechunk` | Decode HTTP chunked transfer-coding in place over [b, b+len). | `http.hpp` |
| `starts_ci` | Does the string s start with pfx, the case ignored? | `http.hpp` |
| `parse_url` | Parse "[http(s)://]host[:port][/path]". | `http.hpp` |
| `resolve_redirect` | Resolve a redirect Location against the current absolute URL `base`, into out[cap]. | `http.hpp` |
| `Transport` | A thin send/recv/close seam so HttpClient is oblivious to http vs https. | `http.hpp` |
| `tp_open` | Open the connection (and TLS handshake if requested). | `http.hpp` |
| `tp_send` | Open the connection (and TLS handshake if requested). | `http.hpp` |
| `tp_recv` | Open the connection (and TLS handshake if requested). | `http.hpp` |
| `kapi_tcp_recv` | Open the connection (and TLS handshake if requested). | `http.hpp` |
| `tp_close` | Open the connection (and TLS handshake if requested). | `http.hpp` |
| `kapi_tcp_close` | Open the connection (and TLS handshake if requested). | `http.hpp` |
| `HttpResponse` | A parsed HTTP response. | `http.hpp` |
| `HttpClient` | Reusable HTTP/1.1 client. | `http.hpp` |

`netkit/httpc.h`

httpc.h -- a minimal HTTP/1.0 client for Onyx apps, layered on the ABI v21 TCP socket calls (kapi_tcp_connect/send/recv/close + kapi_net_status).

Design choices for the Onyx user model:

```
  * Header-only (static inline) -- just #include it.
  * NO dynamic allocation (there is no user malloc): the CALLER provides the
    response buffer. Everything else lives on the app stack. So all memory is in
    the app's own address space (static .bss @ 8 GB / stack @ 16 GB); the TCP
    kapis copy between that buffer and the kernel socket buffers.
  * HTTP/1.0 + "Connection: close": the server closes the socket when done, so we
    just read to EOF -- no chunked-transfer decoding needed (1.0 never chunks).
  * Plain HTTP only (port 80 default) -- there is no TLS in the stack yet.
  * Blocking: polls the non-blocking kapi_tcp_recv with a timeout; fine for the
    occasional fetch (it briefly stalls the calling app, like an IRC connect).
```

```
typedef struct
{
    int   status;       // HTTP status code (200, 404, ...), 0 if unparsed
    int   ok;       // 1 if status is 2xx
    char *body;     // points INTO the caller's buffer (start of the body)
    int   body_len;     // number of body bytes in the buffer
    int   total_len;    // total bytes received (headers + body)
    int   truncated;    // 1 if the response did not fit in the buffer
} http_response;
```

## internals (hc__*)

Append a non-negative integer in decimal.

Parse "[http://]host[:port][/path]" into host / port / path (path defaults "/").

## public API

Perform an HTTP request. method = "GET"/"POST"/...; xheaders = extra header lines ("Key: Value\r\n" each) or 0; body/body_len = request body or 0. The response (headers + body) is read into buf[cap]; resp (if non-0) is filled in, with resp->body pointing into buf. Returns the body length (>=0), or <0 on error (-1 bad URL, -2 connect failed).

```
int http_request (const char *method, const char *url,
                  const char *xheaders, const char *body, unsigned body_len,
                  char *buf, unsigned cap, http_response *resp);
int http_get (const char *url, char *buf, unsigned cap, http_response *r);
int http_post (const char *url, const char *body, unsigned len,
                   char *buf, unsigned cap, http_response *r);
```

`netkit/ftpfs.h`

ftpfs.h -- talk to /bin/ftpfs (the FTP: / FTPS: file-system provider) and read its list of remembered servers. Header-only, no libc (C and C++, freestanding apps and newlib tools).

```
  ftpfs_login ("ftp.example.com", "me", "secret", remember)  -> 1 if delivered
      hand a login over IPC (starting ftpfs if needed), so the password never has to be
      written into an FTP: path. remember = also save it in SD:/etc/ftpfs.ini.
  ftpfs_login_site (&site, remember)   same, with the port / FTPS / start folder too
      (the File Viewer's Connect dialog: they pre-fill the form next time).
```

```
  ftpfs_forget ("ftp.example.com")     drop a login (memory + the file).
  ftpfs_load_sites (sites, max)        the remembered servers (the Connect dialog's list).
```

SD:/etc/ftpfs.ini holds one line per server (or .ini sections, see ftpfs_load_sites):

```
    host user hexpass [port tls folder]        ("-" = empty user / password)
```

hexpass = the password XOR a fixed key, in hex: OBFUSCATED, NOT ENCRYPTED. IPC: service "ftpfs"; message type 1 = "host\0user\0pass\0[port\0tls\0folder\0]", 2 = the same + remember, 3 = "host\0" forget (see user/BinUtils/ftpfs.cpp).

```
#define FTPFS_SITES_FILE    "SD:/etc/ftpfs.ini"
#define FTPFS_MAXSITES      16

struct ftpfs_site
{
    char host[128], user[64], pass[64], port[8], folder[128];
    int  tls;           // 1 = FTPS
};

void ftpfs_obf_hex (const char *in, char *out, int cap);    // a password -> its hexpass (XOR a fixed key, in hex: obfuscated, not encrypted) in out (cap bytes, cut to fit)
void ftpfs_unobf_hex (const char *in, char *out, int cap);  // a hexpass (ftpfs_obf_hex) -> the password in out (cap bytes, cut to fit)
```

Parse one file line; 1 if it is a server.

```
int ftpfs_parse_site (const char *line, struct ftpfs_site *s);
```

Format one file line (with '\n'); returns its length.

```
int ftpfs_format_site (const struct ftpfs_site *s, char *out, int cap);
```

A text file saved by a Windows editor: drop a UTF-8 BOM, turn UTF-16 (Notepad) into 8-bit. In place; returns the new length.

```
int ftpfs_text_fix (char *b, int n);
```

The remembered servers, in file order. Returns how many. Also reads the same data written by hand in .ini style:

```
    [ftp.example.com]
    user = me
    password = secret        (plain; or hexpass = <obfuscated>)
    port = 21   tls = 1   folder = /www      (one per line, all optional)
```

```
int ftpfs_load_sites (struct ftpfs_site *sites, int max);
int ftpfs_login (const char *host, const char *user, const char *pass, int remember);
int ftpfs_login_site (const struct ftpfs_site *s, int remember);
int ftpfs_forget (const char *host);
```

**netkit/http.hpp**

http.hpp -- a small reusable HTTP/1.1 client class for Onyx user apps.

Built on the ABI v21 TCP socket kapis (kapi_tcp_connect/send/recv/close + kapi_net_status). Aimed at REST / web-API clients: custom request headers (Authorization, Accept, ...), a request body with a chosen Content-Type, and a parsed response (status code + headers + body, with chunked transfer decoded).

Design (matches the Onyx user model, like httpc.h / uikit.hpp):

```
  * Header-only, freestanding C++ -- works in EVERY app (integer-only GUI apps and
    newlib apps alike). It does NOT use new/delete, the STL, or libc; only kapi and
    its own byte-loop helpers. Compile it into one translation unit (one app).
  * The CALLER provides the response buffer (char buf[N]); the response body/headers
    point INTO it. No dynamic allocation. `truncated` flags an overflow.
  * Connection: close -- one request per connection; we read until the server closes
    (or Content-Length / chunked tells us the body is complete), then parse.
  * Blocking: polls the non-blocking kapi_tcp_recv with a timeout (briefly stalls the
    calling app, like an IRC connect). Fine for occasional API calls.
```

HTTPS: NetSurf and friends get HTTPS from an external TLS stack (libcurl+OpenSSL); Onyx has no TLS yet. This class RECOGNIZES https:// and returns HTTP_ERR_HTTPS so call sites are already HTTPS-shaped. When mbedTLS lands, only the transport helpers (connect/send/recv) below change -- the public API stays identical.

```
  Example (REST):
    char buf[32*1024];
    HttpClient api;
    api.user_agent("myapp/1.0").bearer(token).accept("application/json");
    HttpResponse r = api.get("http://api.example.com/v1/items", buf, sizeof buf);
    if (r.ok()) { /* parse r.body (NUL-terminated), r.body_len */ }
    HttpResponse p = api.post_json("http://api.example.com/v1/items",
                                   "{\"name\":\"x\"}", buf, sizeof buf);
```

Define ONYX_HTTP_TLS (and link the mbedTLS libs) BEFORE including this header to enable https://. Without it the class is fully freestanding and https:// returns HTTP_ERR_HTTPS. The TLS glue (transport) lives in user/Libs/tls/onyx_tls.hpp.

Transport / protocol errors are reported as a NEGATIVE HttpResponse::status.

```
enum HttpError
{
    HTTP_ERR_BAD_URL = -1,  // could not parse the URL / empty host
    HTTP_ERR_HTTPS   = -2,  // https:// requested but TLS not compiled in (no ONYX_HTTP_TLS)
    HTTP_ERR_NO_NET  = -3,  // the network link is down
    HTTP_ERR_CONNECT = -4,  // DNS or TCP connect failed
    HTTP_ERR_SEND    = -5,  // send() failed
    HTTP_ERR_TIMEOUT = -6,  // no response within the timeout
    HTTP_ERR_EMPTY   = -7,  // connection closed with no data
    HTTP_ERR_TLS     = -8,  // TLS handshake / protocol error
};

namespace http_detail
{
```

The length of the string s (0 for a null pointer).

```
int  slen (const char *s);
```

c in lower case ('A'..'Z' only).

```
char lc (char c);
```

Append s (or n bytes of s) to d[*o], never overflowing cap; keeps d NUL-terminated.

```
void cat (char *d, int cap, int *o, const char *s);
void catn (char *d, int cap, int *o, const char *s, int n);
```

```
inline void cati (char *d, int cap, int *o, long v) // non-negative decimal
    {
char t[20]; int n = 0;
}
```

Case-insensitive lookup of a header in [hdr, hdr+hlen). Returns the value start (trimmed) + its length via *vlen, or 0 if absent.

```
const char *find (const char *hdr, int hlen, const char *name, int *vlen);
```

The decimal number at the start of [s, s+n) (its leading digits, no sign) -> its value, 0 if there is no digit.

```
int parse_int (const char *s, int n);
```

Case-insensitive: does [s,s+n) contain the word w?

```
bool contains_ci (const char *s, int n, const char *w);
```

Decode HTTP chunked transfer-coding in place over [b, b+len). Returns new length.

```
int dechunk (char *b, int len);
```

Does the string s start with pfx, the case ignored?

```
bool starts_ci (const char *s, const char *pfx);
```

Parse "[http(s)://]host[:port][/path]". Sets *https, *port, host, path("/").

```
bool parse_url (const char *url, char *host, int hcap, unsigned *port, char *path, int pcap,
bool *https);
```

Resolve a redirect Location against the current absolute URL `base`, into out[cap]. Handles absolute (http(s)://), protocol-relative (//host/...), absolute-path (/...) and path-relative Locations. Returns false on parse error.

```
bool resolve_redirect (const char *base, const char *loc, char *out, int cap);
}
```

## transport: plain TCP, or TLS (mbedTLS) when ONYX_HTTP_TLS is defined

A thin send/recv/close seam so HttpClient is oblivious to http vs https. recv() keeps the kapi_tcp_recv convention: >0 bytes, 0 = nothing yet (poll), <0 = closed.

```
struct Transport
{
    int  sock;  // kapi TCP handle
    bool tls;
#ifdef ONYX_HTTP_TLS
    onyx_tls::Session ssl;
#endif
};

namespace http_detail
{
```

Open the connection (and TLS handshake if requested). 0 ok, else an HttpError; on connect failure *raw gets the raw kapi_tcp_connect code (for diagnostics).

```
int tp_open (Transport &t, const char *host, unsigned port, bool tls, int *raw);
int tp_send (Transport &t, const void *b, int n);
inline int tp_recv (Transport &t, void *b, int n)   // >0 / 0 = none yet / <0 = closed
    {
return kapi_tcp_recv (t.sock, b, (unsigned) n);
} inline void tp_close (Transport &t);
kapi_tcp_close (t.sock);
}
}
```

A parsed HTTP response. body/hdr point INTO the caller's buffer; body is NUL-terminated. status is the HTTP code (e.g. 200, 404) or a negative HttpError.

```
struct HttpResponse
{
    int         status;
    bool        truncated;  // the response did not fit in the buffer
    const char *body;
    int         body_len;
    const char *hdr;    // header block (between the status line and the blank line)
    int         hdr_len;
    int         net_err;    // raw kapi_tcp_connect code when status==HTTP_ERR_CONNECT
                // (-1 no net, -2 too many sockets, -3 DNS fail, -5 connect); else 0

    bool ok (void)       const { return status >= 200 && status < 300; }
    bool is_error (void) const { return status < 0; }   // transport/protocol error

    // Copy a response header value (case-insensitive) into out[cap], NUL-terminated.
    // Returns the value length (may exceed cap-1 if truncated), or -1 if absent.
    int header (const char *name, char *out, int cap) const
    {
        int vl = 0;
        const char *v = http_detail::find (hdr, hdr_len, name, &vl);
        if (!v) { if (cap > 0) out[0] = '\0'; return -1; }
        int n = vl < cap - 1 ? vl : cap - 1;
        for (int i = 0; i < n; i++) out[i] = v[i];
        if (cap > 0) out[n] = '\0';
        return vl;
    }
};
```

Reusable HTTP/1.1 client. Configure default headers once (chainable), then issue any number of requests. Stateless across calls except for the default headers.

```
class HttpClient
{
public:
    HttpClient (void) : m_hdrs_len (0), m_timeout (15000), m_ua ("Onyx/1.0"), m_prog (0),
m_progCtx (0)
    {
        m_hdrs[0] = '\0';
    }

    // --- configuration (chainable) ----------------------------------------
    HttpClient &user_agent (const char *ua) { m_ua = ua ? ua : "Onyx/1.0"; return *this; }
    HttpClient &timeout_ms (unsigned ms)    { m_timeout = ms; return *this; }
    // Told the bytes received so far (headers included) while a response comes in: a
download's
    // progress. fn returns false to stop (the response then ends there, truncated).
```

```
    HttpClient &progress (bool (*fn) (void *ctx, long got), void *ctx) { m_prog = fn;
m_progCtx = ctx; return *this; }
    void        reset_headers (void)        { m_hdrs_len = 0; m_hdrs[0] = '\0'; }

    // Add a default header line ("Name: Value") sent with every request.
    HttpClient &header (const char *name, const char *value)
    {
        using namespace http_detail;
        cat (m_hdrs, (int) sizeof m_hdrs, &m_hdrs_len, name);
        cat (m_hdrs, (int) sizeof m_hdrs, &m_hdrs_len, ": ");
        cat (m_hdrs, (int) sizeof m_hdrs, &m_hdrs_len, value);
        cat (m_hdrs, (int) sizeof m_hdrs, &m_hdrs_len, "\r\n");
        return *this;
    }
    HttpClient &bearer (const char *token)  // Authorization: Bearer <token>
    {
        using namespace http_detail;
        cat (m_hdrs, (int) sizeof m_hdrs, &m_hdrs_len, "Authorization: Bearer ");
        cat (m_hdrs, (int) sizeof m_hdrs, &m_hdrs_len, token);
        cat (m_hdrs, (int) sizeof m_hdrs, &m_hdrs_len, "\r\n");
        return *this;
    }
    HttpClient &accept (const char *type) { return header ("Accept", type); }

    // --- requests ------------------------------------------------------------
    HttpResponse get (const char *url, char *buf, int cap)
    { return send ("GET", url, 0, 0, 0, buf, cap); }
    HttpResponse del (const char *url, char *buf, int cap)
    { return send ("DELETE", url, 0, 0, 0, buf, cap); }
    HttpResponse post (const char *url, const char *ctype, const void *body, int len,
                       char *buf, int cap)
    { return send ("POST", url, ctype, body, len, buf, cap); }
    HttpResponse put (const char *url, const char *ctype, const void *body, int len,
                      char *buf, int cap)
    { return send ("PUT", url, ctype, body, len, buf, cap); }
    HttpResponse patch (const char *url, const char *ctype, const void *body, int len,
                        char *buf, int cap)
    { return send ("PATCH", url, ctype, body, len, buf, cap); }

    // JSON convenience (Content-Type: application/json); `json` is a C string.
    HttpResponse post_json (const char *url, const char *json, char *buf, int cap)
    { return send ("POST", url, "application/json", json, http_detail::slen (json), buf,
cap); }
    HttpResponse put_json (const char *url, const char *json, char *buf, int cap)
    { return send ("PUT", url, "application/json", json, http_detail::slen (json), buf, cap);
}

    // The general request, following up to 8 HTTP 3xx redirects (resolving relative
    // Locations). 301/302/303 switch a non-GET to GET (curl's default); 307/308 keep
    // the method + body. Returns the FINAL response (or the last 3xx if it has no
    // Location / the hop limit is hit). body/hdr point into buf[cap]; <0 = HttpError.
    HttpResponse send (const char *method, const char *url, const char *ctype,
                       const void *body, int len, char *buf, int cap)
    {
        using namespace http_detail;
        char cur[2048]; int o = 0; cur[0] = '\0';
        cat (cur, (int) sizeof cur, &o, url);

        HttpResponse r = send_once (method, cur, ctype, body, len, buf, cap);
```

```
        for (int hop = 0; hop < 8; hop++)
        {
            if (r.status < 300 || r.status >= 400 || r.status == 304)
                return r;           /* not a redirect we follow */
            char loc[2048];
            if (r.header ("Location", loc, (int) sizeof loc) <= 0)
                return r;           /* 3xx without Location: hand it back */
            char next[2048];
            if (!resolve_redirect (cur, loc, next, (int) sizeof next))
                return r;
            o = 0; cur[0] = '\0';
            cat (cur, (int) sizeof cur, &o, next);
            if (r.status != 307 && r.status != 308)
            { method = "GET"; body = 0; len = 0; ctype = 0; }   /* 301/302/303 -> GET */
            r = send_once (method, cur, ctype, body, len, buf, cap);
        }
        return r;                   /* too many redirects */
    }

    // One request, no redirect handling. method = "GET"/"POST"/...; ctype/body optional.
    // The response is read into buf[cap]; body/hdr point into it. Negative status =
HttpError.
    HttpResponse send_once (const char *method, const char *url, const char *ctype,
                const void *body, int len, char *buf, int cap)
    {
        using namespace http_detail;

        HttpResponse r;
        r.status = 0; r.truncated = false;
        r.body = buf; r.body_len = 0; r.hdr = buf; r.hdr_len = 0; r.net_err = 0;
        if (cap < 16) { r.status = HTTP_ERR_BAD_URL; return r; }

        char host[160], path[1024]; unsigned port; bool https;
        if (!parse_url (url, host, (int) sizeof host, &port, path, (int) sizeof path,
&https))
        { r.status = HTTP_ERR_BAD_URL; return r; }
#ifndef ONYX_HTTP_TLS
        if (https) { r.status = HTTP_ERR_HTTPS; return r; }
#endif

        char ip[40];
        if (!kapi_net_status (ip, sizeof ip)) { r.status = HTTP_ERR_NO_NET; return r; }

        Transport tp;
        { int orc = tp_open (tp, host, port, https, &r.net_err); if (orc != 0) { r.status =
orc; return r; } }

        // Build the request line + headers.
        char req[2048]; int o = 0; req[0] = '\0';
        cat (req, (int) sizeof req, &o, method);
        cat (req, (int) sizeof req, &o, " ");
        cat (req, (int) sizeof req, &o, path);
        cat (req, (int) sizeof req, &o, " HTTP/1.1\r\nHost: ");
        cat (req, (int) sizeof req, &o, host);
        if (port != 80) { cat (req, (int) sizeof req, &o, ":"); cati (req, (int) sizeof req,
&o, (long) port); }
        cat (req, (int) sizeof req, &o, "\r\nUser-Agent: ");
        cat (req, (int) sizeof req, &o, m_ua);
        cat (req, (int) sizeof req, &o, "\r\nConnection: close\r\n");
```

```
        catn (req, (int) sizeof req, &o, m_hdrs, m_hdrs_len);
        if (ctype && ctype[0])
        { cat (req, (int) sizeof req, &o, "Content-Type: "); cat (req, (int) sizeof req, &o,
ctype);
          cat (req, (int) sizeof req, &o, "\r\n"); }
        if (body && len > 0)
        { cat (req, (int) sizeof req, &o, "Content-Length: "); cati (req, (int) sizeof req,
&o, len);
          cat (req, (int) sizeof req, &o, "\r\n"); }
        cat (req, (int) sizeof req, &o, "\r\n");

        if (tp_send (tp, req, o) < 0)
        { tp_close (tp); r.status = HTTP_ERR_SEND; return r; }
        if (body && len > 0 && tp_send (tp, body, len) < 0)
        { tp_close (tp); r.status = HTTP_ERR_SEND; return r; }

        // Read until the server closes, the buffer fills, or we idle past the timeout.
        int total = 0;
        unsigned start = kapi_get_ticks ();
        for (;;)
        {
            if (total >= cap - 1) { r.truncated = true; break; }
            int n = tp_recv (tp, buf + total, cap - 1 - total);
            if (n > 0)
            {
                total += n; start = kapi_get_ticks ();
                if (m_prog && !m_prog (m_progCtx, total)) { r.truncated = true; break; }
            }
            else if (n == 0)
            {
                if ((kapi_get_ticks () - start) * 10 > m_timeout)       // (ticks: HZ = 100)
                {
                    if (total == 0)
                    { tp_close (tp); r.status = HTTP_ERR_TIMEOUT; return r; }
                    break;
                }
                kapi_msleep (5);
            }
            else break; // connection closed -> response complete
        }
        buf[total] = '\0';
        tp_close (tp);
        if (total == 0) { r.status = HTTP_ERR_EMPTY; return r; }

        // Status line: "HTTP/1.x NNN reason".
        int i = 0;
        while (i < total && buf[i] != ' ') i++;
        int code = 0;
        if (i < total) { i++; while (i < total && buf[i] >= '0' && buf[i] <= '9') { code =
code * 10 + (buf[i] - '0'); i++; } }
        r.status = code;

        // Header block = after the status-line LF, up to the blank line.
        int hs = 0; while (hs < total && buf[hs] != '\n') hs++; if (hs < total) hs++;
        int he = -1;
        for (int k = hs; k + 3 < total; k++)
            if (buf[k] == '\r' && buf[k+1] == '\n' && buf[k+2] == '\r' && buf[k+3] == '\n') {
he = k; break; }
        if (he < 0) { r.hdr = buf + hs; r.hdr_len = total - hs; r.body = buf + total;
```

```
r.body_len = 0; return r; }
        r.hdr = buf + hs; r.hdr_len = he - hs;

        int bstart = he + 4;
        int blen = total - bstart;

        int vl = 0;
        const char *te = find (r.hdr, r.hdr_len, "Transfer-Encoding", &vl);
        if (te && contains_ci (te, vl, "chunked"))
            blen = dechunk (buf + bstart, blen);
        else
        {
            const char *cl = find (r.hdr, r.hdr_len, "Content-Length", &vl);
            if (cl) { int n = parse_int (cl, vl); if (n >= 0 && n < blen) blen = n; }
        }

        r.body = buf + bstart;
        r.body_len = blen;
        buf[bstart + blen] = '\0';  // bstart+blen <= total <= cap-1
        return r;
    }

private:
    char     m_hdrs[1024];  // concatenated default header lines
    int      m_hdrs_len;
    unsigned m_timeout; // idle timeout in ms
    const char *m_ua;
    bool (*m_prog) (void *, long);  // the progress (0: none)
    void *m_progCtx;
};
```